

目录

1 问题描述与分析	3
1.1 前提分析	3
1.2 问题描述	3
1.3 功能分析	3
1.4 开发环境	3
2 功能模块分析及数据结构	4
2.1 功能模块分析	4
2.2 数据结构选择	4
2.3 数据库设计	5
2.3.1 概念设计	5
2.3.2 逻辑设计	5
3 系统设计实现描述及其流程图	6
3.1 系统流程图	6
3.2 登录注册功能	7
3.3 录像功能	11
3.4 匹配功能	13
3.5 AI 代码执行	16
3.6 更新 AI 代码	17
3.7 添加 AI 代码	21
3.8 删除 AI 代码	23
3.9 其他设计与实现描述	25
4 系统使用说明和测试	27
4.1 启动流程	27
4.2 登录注册功能测试	27
4.3 录像功能测试	29
4.4 匹配功能测试	30
4.5 AI 代码执行功能测试	31
4.6 删除 AI 功能测试	33
4.7 添加 AI 功能测试	34
4.8 更新 AI 功能测试	35

5 问题及解决办法	38
5.1 JWT - token 如何解决跨域问题:	38
5.2 Java 是如何解析的 JSON 的:	38
5.3 Redis 是如何解决缓存:	38
6 项目总结	40
参考文献	41

1 问题描述与分析

1.1 前提分析

游戏发展至今,划分出了休闲益智、体育竞速、动作冒险、飞行射击、经营策略、角色扮演等游戏类型。在众多的游戏分类中,在线匹配游戏是发展较早的游戏类型之一,在电子游戏的历史演变过程中扮演了重要的角色,留下很多经典的代表作,如“王者”、“英雄联盟”等。随着 Web 的普及,在线匹配游戏在 Web 平台也应用广泛,通过用户键盘控制,能给用户带来更畅快的体验。本文基于 Web 平台,分析相关开发技术,设计实现一款在线匹配游戏。

1.2 问题描述

在线匹配游戏首先需要实现在线多人匹配,其次需要具有用户身份验证和安全性功能以保护用户数据的功能。该系统还可以有以下内容:可以人机对战,人人对战,AI 对战,查看对局回放(所有人的在线对局记录),查看排行榜(每个人都有一个天梯分,按照天梯分从高到低排序),编写 AI 代码(基于 dfs 搜索)。该设计主要分为实验的系统分析,具体的算法实现,以及最终的实验测试,最终完成实验设计。该在线匹配游戏的游戏规则相对简单,无需特别的技巧,也不需要较长的时间就能完成一局游戏。但写 AI 代码需要有很高的算法水平,要对 dfs 非常了解才可以正常控制蛇的移动。

1.3 功能分析

- (1) 用户身份验证的功能,因此只有经过授权的用户才能访问和修改个人 AI 代码。如何才能跨设备登录,跨域问题又如何解决。如何加密可以防止暴力攻击。
- (2) 匹配功能,当多人匹配时,如何选择两个用户匹配到一起,匹配算法如何才最优。
- (3) AI 代码,如何让蛇具有提前预判功能,可以让蛇活的更久。
- (4) 录像功能,如何保持双方的操作,双方操作是如何再次体现出来的。
- (5) AI 代码的添加,删除,更新。又是如何实现,添加时,代码高亮又是如何设计。

1.4 开发环境

Java: Java 1.8

Vue: Vue3.0

Spring: SpringBoot 2.7.5

MySQL : 8.0.31

MyBatisPlus: 3.5.3

2 功能模块分析及数据结构

2.1 功能模块分析

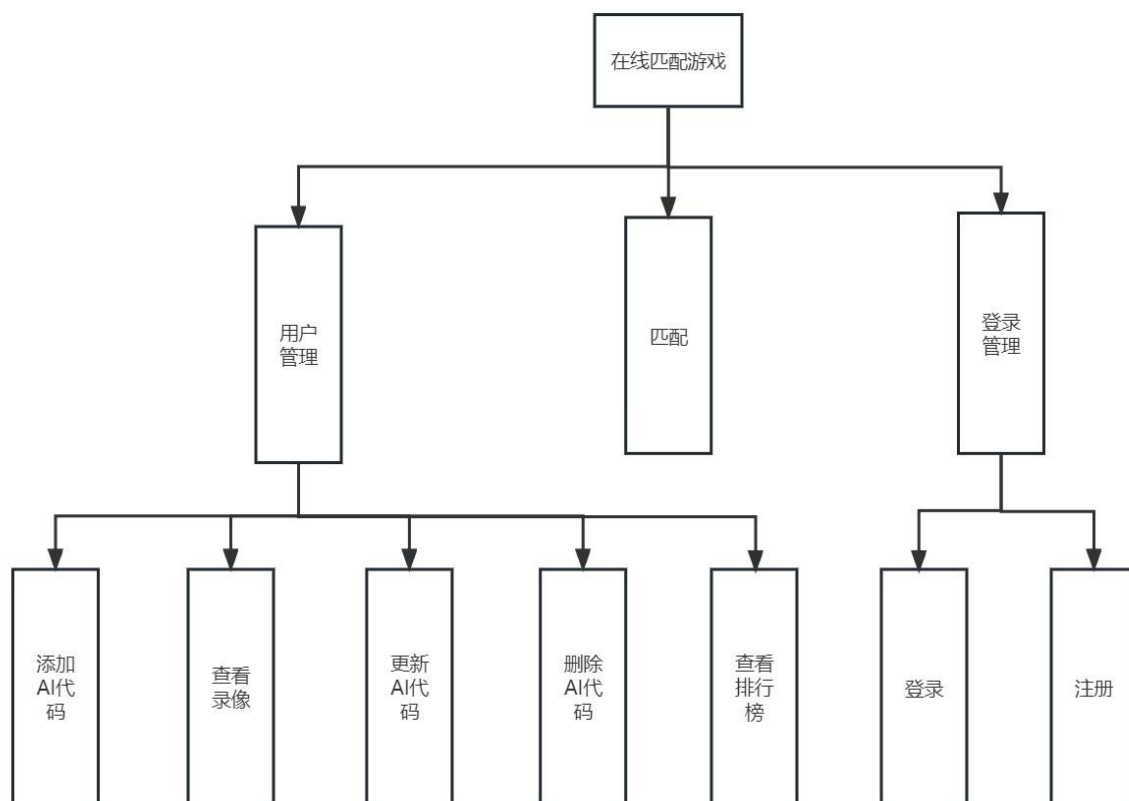


图 2.1 系统功能模块图

该图表示了在线匹配游戏有三大模块, 分别是用户管理, 匹配系统, 登录管理。用户管理又分为添加 AI 代码, 查看录像, 更新 AI 代码, 删除 AI 代码, 查看排行榜。登录管理分为登录, 注册功能。

2.2 数据结构选择

为什么要用数据结构(容器) List 和 Map:

当我们需要保存一组类型相同的数据的时候, 我们应该是用一个容器来保存, 这个容器可以是数组, 但是, 使用数组存储对象具有一定的弊端, 因为我们在实际开发中, 存储的数据的类型是多种多样的, 于是, 就出现了“集合”, 集合同样也是用来存储多个数据的。数组的缺点是一旦声明之后, 长度就不可变了; 同时, 声明数组时的数据类型也决定了该数组存储的数据的类型; 而且, 数组存储的数据是有序的、可重复的, 特点单一。但是集合提高了数据存储的灵活性, Java 集合不仅可以用来存储不同类型不同数量的对象, 还可以保存具有映射关系的数据。对于查询操作来说, 由

于用的是 MybatisPlus，MybatisPlus 查询用的 selectList 这个返回的是一个 List，List：存储的元素是有序的、可重复的。其他操作都是用 Map 实现的，Map：使用键值对（key-value）存储，类似于数学上的函数 $y=f(x)$ ，“x”代表 key，“y”代表 value，key 是无序的、不可重复的，value 是无序的、可重复的，每个键最多映射到一个值

2.3 数据库设计

2.3.1 概念设计

- （1）用户实体对象。主要包含了用户 ID、用户名、密码、头像、天梯分等；
- （2）对局记录实体对象。包括书籍的具体类别、书籍名称、图书馆中的数量、简要信息描述等；
- （3）AI 代码 实体对象。包括 ID、用户 ID、标题、描述、代码、天梯分、创建时间、修改时间等；

2.3.2 逻辑设计

概念设计完成后即可规定具体的存放数据设定，并明确路基数据模型。数据模型可帮助系统设计人员确定数据的详细信息，基于概念数据派生对象，具有独立于数据存储、技术的特点。其设计的常见步骤包括实体主键确定、实体对应属性查询、实体关系确定以及多对多关系解决。

具体用户 user 表：对用户数据字段进行描述与约束；对局信息 record 表：对局记录字段描述与约束；AI 代码信息 bot 表：AI 代码相关字段描述与约束；

3 系统设计实现描述及其流程图

3.1 系统流程图

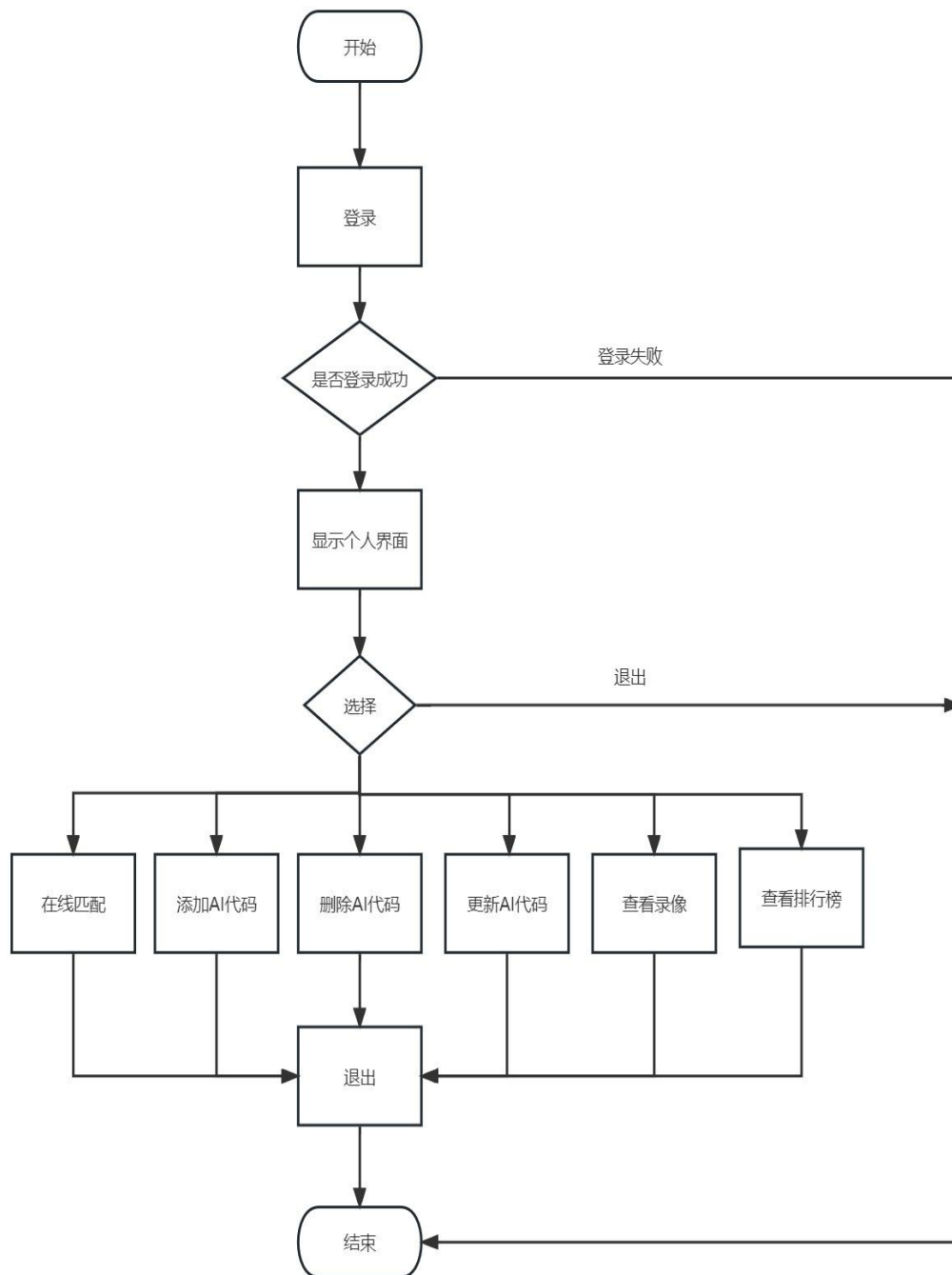


图 3.1 系统功能模块图

3.2 登录注册功能

功能描述：用户通过输入用户名和密码注册，注册完成后登录。

实现与描述：首先需要连接数据库的实体类 user 如下：

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    @TableId(type = IdType.AUTO)
    private Integer id;
    private String username;
    private String password;
    private String photo;
    private Integer rating;
}
```

解释：User 类对应数据库中的 user 表 @TableId(type = IdType.AUTO) 表示 id 自增
@Data @NoArgsConstructor @AllArgsConstructor 三个注解表示 lombok 这个仓库通过反射机制
自动生成 get set toString() 方法。

然后配置一下 application.properties

```
# 端口号
server.port=3000
# 数据库用户和密码
spring.datasource.username=root
spring.datasource.password=123456
spring.datasource.url=jdbc:mysql://localhost:3306/kob?serverTimezone=Asia/Shanghai
&useUnicode=true&characterEncoding=utf-8
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

然后写一个 UserMapper 接口用于使用 MybatisPlus

```
@Mapper
public interface UserMapper extends BaseMapper<User> {
}
```

然后写三个 service, InfoService, LoginService, RegisterService 已经他们的实现方式
InfoServiceImpl, LoginService, RegisterService

```
public interface InfoService {
    public Map<String, String> getinfo();
}

public interface LoginService {
    public Map<String, String> getToken(String username, String password);
}
```

```

    }

    public interface RegisterService {

        public Map<String, String> register(String username, String password, String
confirmedPassword);

    }

    // Service 代码简单, 就是通过传参, 返回 Map。具体还是要看 impl
    @Service
    public class LoginServiceImpl implements LoginService {

        @Autowired
        private AuthenticationManager authenticationManager;

        @Override
        public Map<String, String> getToken(String username, String password) {
            UsernamePasswordAuthenticationToken authenticationToken =
                new UsernamePasswordAuthenticationToken(username, password);

            Authentication authenticate =
authenticationManager.authenticate(authenticationToken); // 登录失败自己处理
            UserDetailsImpl loginuser = (UserDetailsImpl) authenticate.getPrincipal();
            User user = loginuser.getUser();
            String jwt = JwtUtil.createJWT(user.getId().toString());
            Map<String, String> map = new HashMap<>();
            map.put("error_message", "success");
            map.put("token", jwt);
            return map;
        }
    }
}

```

通过调用 JWT-token 的类来判断, 登录是否成功, 如果成功会创建一个 token, 然后将这个 token 返回给前端, 前端可以通过 ajax 解析。并保留到本地浏览器用于下一次登录进行验证。

```

@Service
public class RegisterServiceImpl implements RegisterService {

    @Autowired
    private UserMapper userMapper;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Override

```

```

    public Map<String, String> register(String username, String password, String
confirmedPassword) {
        Map<String, String> map = new HashMap<>();

        if (username == null) {
            map.put("error_message", "用户名不能为空");
            return map;
        }
        if (password == null || confirmedPassword == null) {
            map.put("error_message", "密码不能为空");
            return map;
        }
        username = username.trim();
        if (username.length() == 0) {
            map.put("error_message", "用户名不能为空");
            return map;
        }
        if (password.length() == 0 || confirmedPassword.length() == 0) {
            map.put("error_message", "密码不能为空");
            return map;
        }
        if (username.length() > 100) {
            map.put("error_message", "用户名长度不能大于 100");
            return map;
        }
        if (password.length() > 100 || confirmedPassword.length() > 100) {
            map.put("error_message", "密码长度不能大于 100");
            return map;
        }
        if (!password.equals(confirmedPassword)) {
            map.put("error_message", "两次密码不一致");
            return map;
        }
        QueryWrapper<User> queryWrapper = new QueryWrapper<User>();
        queryWrapper.eq("username", username);
        List<User> users = userMapper.selectList(queryWrapper);
        if (!users.isEmpty()) {
            map.put("error_message", "用户名已存在");

```

```

        return map;
    }

    String encodePassword = passwordEncoder.encode(password);
    String photo =
"https://cdn.acwing.com/media/user/profile/photo/134647_lg_70a6fa9146.jpg";
    User user = new User(null, username, encodePassword, photo, 1500);

    userMapper.insert(user);
    map.put("error_message", "success");
    return map;
}
}

```

RegisterServiceImpl 主要是判断前端传过来的信息 username, password, confirmedPassword。我们进行判断是否合法,比如用户名不能为空,密码不能为空,用户名长度不能大于 100,密码长度不能大于 100,两次密码不一致等等。我们通过 MyBatisPlus 的 selectList 方法获取到一个 List,如果 List 不为空,说明用户名已存在。把所有情况判断完之后,在 userMapper 里面插入新的数据并返回给前端信息,即 map.put("error_message", "success");

最后写三个 Controller, InfoController, LoginController, RegisterController,在这三个 Controller 加入 @RestController 注解首先他是一个类似于@Controller 和@Service 的构造型注解,能够让类被组件扫描功能发现。但是,与 REST 最相关在于@RestController 会告诉 Spring,控制器中所有的处理器方法的返回值都要直接写入响应体中,而不是将值放到模型中并传递给一个视图以便于渲染。

```

@RestController
public class RegisterController {

    @Autowired
    private RegisterService registerService;

    @PostMapping("/user/account/register/")
    public Map<String, String> register(@RequestParam Map<String, String> map) {
        String username = map.get("username");
        String password = map.get("password");
        String confirmedPassword = map.get("confirmedPassword");
        return registerService.register(username, password, confirmedPassword);
    }
}

```

@RestController

```

public class LoginController {

    @Autowired
    private LoginService loginService;

    @PostMapping("/user/account/token/")
    public Map<String, String> getToken(@RequestParam Map<String, String> map) {
        String username = map.get("username");
        String password = map.get("password");

        return loginService.getToken(username, password);
    }
}

```

Controller 层的主要作用是处理业务逻辑和数据校验等，为前端提供合适的数据和响应格式，并将请求转发到具体的 Service 层进行业务处理。在 Controller 层中，我们通常会定义一些 HTTP 请求的路由映射，并将这些路由映射到服务层的具体方法上。

3.3 录像功能

功能描述：录像功能可以将对局结束再次重现出来。

实现与描述：将匹配成功的双方 id, step, map 地图信息，胜负情况放到数据库中，这样我们可以通过数据库中的信息，把当时的对局重现出来。即我们需要一个 record 表。

```

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Record {
    @TableId(type = IdType.AUTO)
    private Integer id;
    private Integer aId;
    private Integer aSx;
    private Integer aSy;
    private Integer bId;
    private Integer bSx;
    private Integer bSy;
    private String aSteps;
    private String bSteps;
    private String map;
}

```

```

        private String loser;
        @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai")
        private Date createtime;
    }

```

通过这个 Record 这个类来对应数据库中的 record, 同时我们还需要开始游戏的接口和蛇移动的接口。

@Mapper

```

public interface RecordMapper extends BaseMapper<Record> {
}

```

这个通过 @Mapper 注解, 使用 MybaitPlus。

@Service

```

public class ReceiveBotMoveServiceImpl implements ReceiveBotMoveService {
    @Override
    public String receiveBotMove(Integer userId, Integer direction) {
        System.out.println("receive bot move" + userId + " " + direction);

        if (WebSocketServer.users.get(userId) != null) {
            Game game = WebSocketServer.users.get(userId).game;

            if (game != null) {
                if (game.getPlayerA().getId().equals(userId)) { // 人工
                    game.setNextStepA(direction);
                } else if (game.getPlayerB().getId().equals(userId)) {
                    game.setNextStepB(direction);
                }
            }
        }

        return "receive bot move success";
    }
}

```

这个通过传进来的两个参数 userId 和 direction 方向, 如果 userId 不是空, 则取出地图, 将下一个操作变成 direction。

@Service

```

public class StartGameServiceImpl implements StartGameService {
    @Override
    public String startGame(Integer aId, Integer aBotId, Integer bId, Integer bBotId)
    {
        System.out.println("startGame" + aId + " " + bId);
        WebSocketServer.startGame(aId, aBotId, bId, bBotId);
        return "start game success";
    }
}

```

```
}
```

开始游戏 Impl, 就是将双方的 id 和 Al_id 传给前端。并返回信息表示成功匹配。

3.4 匹配功能

功能描述: 将在匹配的玩家放到一个匹配池中, 选出两个实力相近的玩家匹配到一起。

实现与描述: 分析一个可能出现读写冲突的问题, 所有对于所有的操作都要加锁,

```
public interface MatchingService {  
    String addPlayer(Integer userId, Integer rating, Integer botId);  
    String removePlayer(Integer userId);  
}
```

有两个 Service, 一个用于添加到匹配池中, 一个从匹配池中取出来。

@Service

```
public class MatchingServiceImpl implements MatchingService {  
  
    public final static MatchingPool matchingPool = new MatchingPool();  
    @Override  
    public String addPlayer(Integer userId, Integer rating, Integer botId) {  
        matchingPool.addPlayer(userId, rating, botId);  
        return "success add player";  
    }  
    @Override  
    public String removePlayer(Integer userId) {  
        matchingPool.removePlayer(userId);  
        return "success remove player";  
    }  
}
```

Remove 就是在匹配池中找到这个用户, 删除他。添加操作, 就是将当前用户的 id, 天梯分, al_id 添加到匹配池中。

@Component

```
public class MatchingPool extends Thread{  
  
    private static List<Player> players = new ArrayList<>();  
    private final ReentrantLock lock = new ReentrantLock();  
  
    private static RestTemplate restTemplate;  
    private final static String startGameUrl =  
    "http://127.0.0.1:3000/pk/start/game/";
```

```

@Autowired
public void setRestTemplate(RestTemplate restTemplate) {
    MatchingPool.restTemplate = restTemplate;
}

public void addPlayer(Integer userId, Integer rating, Integer botId) {
    lock.lock();
    try {
        players.add(new Player(userId, rating, botId, 0));
    } finally {
        lock.unlock();
    }
}

public void removePlayer(Integer userId) {
    lock.lock();
    try {
        List<Player> newPlayers = new ArrayList<>();
        for (Player player : players) {
            if (!player.getUserId().equals(userId)) {
                newPlayers.add(player);
            }
        }
        players = newPlayers;
    } finally {
        lock.unlock();
    }
}

private void increaseWaitingTime() {
    for (Player player : players) {
        player.setWaitingTime(player.getWaitingTime() + 1);
    }
}

private boolean checkMatches(Player a, Player b) {
    int ratingDelta = Math.abs(a.getRating() - b.getRating());
    int waitingTime = Math.min(a.getWaitingTime(), b.getWaitingTime());
    return ratingDelta <= waitingTime * 10;
}

private void sendResult(Player a, Player b) {
    System.out.println("send result" + a + " " + b);
}

```

```

        MultiValueMap<String, String> data = new LinkedMultiValueMap<>();
        data.add("a_id", a.getUserId().toString());
        data.add("a_bot_id", a.getBotId().toString());
        data.add("b_id", b.getUserId().toString());
        data.add("b_bot_id", b.getBotId().toString());
        restTemplate.postForObject(startGameUrl, data, String.class);
    }

    private void matchPlayers() {
        boolean[] used = new boolean[players.size()];
        for (int i = 0; i < players.size(); i++) {
            if (used[i]) continue;
            for (int j = i + 1; j < players.size(); j++) {
                if (used[j]) continue;
                Player a = players.get(i), b = players.get(j);
                if (checkMatches(a, b) && !a.getUserId().equals(b.getUserId())) {
                    used[i] = used[j] = true;
                    sendResult(a, b);
                    break;
                }
            }
        }

        List<Player> newPlayers = new ArrayList<>();
        for (int i = 0; i < players.size(); i++) {
            if (!used[i]) {
                newPlayers.add(players.get(i));
            }
        }

        players = newPlayers;
    }

    @Override
    public void run() {
        while (true) {
            try {
                Thread.sleep(1000);
                lock.lock();
                try {
                    increaseWaitingTime();
                    matchPlayers();
                }
            }
        }
    }

```

```

        } finally {
            lock.unlock();
        }

        } catch (InterruptedException e) {
            e.printStackTrace();
            break;
        }
    }
}

```

由于会出现读写冲突的问题，所有都加了锁，remove 一个用户算法实现是先申请一个 list 遍历整个匹配池，如果不等于当前用户，则添加到 List 中，最后返回这个 list。add 一个用户就是把信息用 add 方法添加进去，就是一个简单的集合添加功能。

3.5 AI 代码执行

功能描述：蛇通过代码控制移动不需要人用键盘输入。

实现与描述：首先分析每个 AI 代码必须要有用户 id, AI 代码，和下一步操作。将这个用一个类来表示

```

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Bot {
    Integer userId;
    String botCode;
    String input;
}

```

然后就是读取 AI 代码的操作。将操作传给一开始的蛇的移动。

```

public class BotPool extends Thread {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();
    private final Queue<Bot> bots = new LinkedList<>();

    public void addBot(Integer userId, String botCode, String input) {
        lock.lock();
        try {
            bots.add(new Bot(userId, botCode, input));

```

```

        condition.signalAll();
    } finally {
        lock.unlock();
    }
}

private void consume(Bot bot) {
    Consumer consumer = new Consumer();
    consumer.startTimeout(2000, bot);
}

@Override
public void run() {
    while (true) {
        lock.lock();
        if (bots.isEmpty()) {
            try {
                condition.await();
            } catch (InterruptedException e) {
                e.printStackTrace();
                lock.unlock();
                break;
            }
        } else {
            Bot bot = bots.remove();
            lock.unlock();
            consume(bot);
        }
    }
}
}

```

多线程有四个实现方式：继承 Thread 类,实现 Runnable 接口,实现 Callable 接口,线程池。这里继承 Thread 重写 run 方法, run 的方法说如果 bots 不为空,就取出下一步操作,然后把锁解开。否则抛出异常。

3.6 更新 AI 代码

功能描述：更新数据库中 bot 的信息,即更新 AI 代码

实现与描述：首先需要连接数据库的实体类 bot 如下：

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Bot {
    @TableId(type = IdType.AUTO)
    private Integer id;
    private Integer userId;
    private String title;
    private String description;
    private String content;
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai")
    private Date createtime;
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai")
    private Date modifytime;
}
```

解释：Bot 类对应数据库中的 bot 表 @TableId(type = IdType.AUTO) 表示 id 自增 @Data @NoArgsConstructor @AllArgsConstructor 三个注解表示 lombok 这个仓库通过反射机制自动生成 get set toString() 方法 @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai") 表示以什么格式表示日期这个表示先年再月最后天 几点几分, 如 2023-06-15 21:15, timezone 表示时间是以亚洲上海为标准。

写一个 Service

```
public interface UpdateService {
    Map<String, String> update(Map<String, String> data);
}
```

然后 实现 Impl

```
@Service
public class UpdateServiceImpl implements UpdateService {

    @Autowired
    private BotMapper botMapper;

    @Override
    public Map<String, String> update(Map<String, String> data) {
        UsernamePasswordAuthenticationToken authenticationToken =
            (UsernamePasswordAuthenticationToken)
                SecurityContextHolder.getContext().getAuthentication();
    }
}
```

```

        UserDetailsImpl loginUser = (UserDetailsImpl)
authenticationToken.getPrincipal();

        User user = loginUser.getUser();
        int bot_id = Integer.parseInt(data.get("bot_id"));
        String title = data.get("title");
        String description = data.get("description");
        String content = data.get("content");
        Map<String, String> map = new HashMap<>();

        if (title == null || title.length() == 0) {
            map.put("error_message", "标题不能为空");
            return map;
        }

        if (title.length() > 100) {
            map.put("error_message", "标题长度不能大于 100");
            return map;
        }

        if (description == null || description.length() == 0) {
            description = "这个用户..., 什么也没留下~";
        }

        if (description.length() > 300) {
            map.put("error_message", "Bot 描述的长度不能大于 300");
            return map;
        }

        if (content == null || content.length() == 0) {
            map.put("error_message", "代码不能为空");
            return map;
        }

        if (content.length() > 10000) {
            map.put("error_message", "代码长度不能超过 10000");
            return map;
        }

```

```

    }

    Bot bot = botMapper.selectById(bot_id);

    if (bot == null) {
        map.put("error_message", "Bot 不存在或已被删除");
        return map;
    }

    if (!bot.getUserId().equals(user.getId())) {
        map.put("error_message", "没有权限删除该 Bot");
        return map;
    }

    Bot new_bot = new Bot(
        bot.getId(),
        user.getId(),
        title,
        description,
        content,
        bot.getCreatetime(),
        new Date()
    );

    botMapper.updateById(new_bot);
    map.put("error_message", "success");
    return map;
}
}

```

先通过前端传过来的 data, 通过 map 的 get 方法获取到 bot_id, title, content 等信息, 然后判断是否合法, 比如: 标题不能为空, 标题长度不能大于 100, 代码不能为空等等, 这个需要特别判断一个是不是当前用户, 即判断一下 Bot 的 userID 是不是与 user 的 ID 是不是一样, 最后将新的 Bot 添加 数据库中。

```

@RestController
public class UpdateController {

    @Autowired
    private UpdateService updateService;

    @PostMapping("/api/user/bot/update")
    public Map<String, String> update(@RequestParam Map<String, String> data) {
        return updateService.update(data);
    }
}

```

```

    }
}

```

前端通过 ajax 调用这个接口传入参数, 后端再返回。

3.7 添加 AI 代码

功能描述: 添加数据库中 bot 的信息, 即添加 AI 代码

实现与描述: 首先需要连接数据库的实体类 bot 如下:

```

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Bot {
    @TableId(type = IdType.AUTO)
    private Integer id;
    private Integer userId;
    private String title;
    private String description;
    private String content;
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai")
    private Date createtime;
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai")
    private Date modifytime;
}

```

解释: Bot 类对应数据库中的 bot 表 @TableId(type = IdType.AUTO) 表示 id 自增 @Data @NoArgsConstructor @AllArgsConstructor 三个注解表示 lombok 这个仓库通过反射机制自动生成 get set toString() 方法 @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai") 表示以什么格式表示日期这个表示先年再月最后天 几点几分, 如 2023-06-15 21:15, timezone 表示时间是以亚洲上海为标准。

写一个 Service

```

public interface AddService {
    Map<String, String> add(Map<String, String> data);
}

```

然后 实现 Impl

```

@Service
public class AddServiceImpl implements AddService {

    @Autowired

```

```

private BotMapper botMapper;

@Override
public Map<String, String> add(Map<String, String> data) {
    UsernamePasswordAuthenticationToken authenticationToken =
        (UsernamePasswordAuthenticationToken)
SecurityContextHolder.getContext().getAuthentication();
    UserDetailsImpl loginUser = (UserDetailsImpl)
authenticationToken.getPrincipal();

    User user = loginUser.getUser();

    String title = data.get("title");
    String description = data.get("description");
    String content = data.get("content");

    Map<String, String> map = new HashMap<>();

    if (title == null || title.length() == 0) {
        map.put("error_message", "标题不能为空");
        return map;
    }

    if (title.length() > 100) {
        map.put("error_message", "标题长度不能大于 100");
        return map;
    }

    if (description == null || description.length() == 0) {
        description = "这个用户..., 什么也没留下~";
    }

    if (description.length() > 300) {
        map.put("error_message", "Bot 描述的长度不能大于 300");
        return map;
    }

    if (content == null || content.length() == 0) {

```

```

        map.put("error_message", "代码不能为空");
        return map;
    }

    if (content.length() > 10000) {
        map.put("error_message", "代码长度不能超过 10000");
        return map;
    }

    Date now = new Date();
    Bot bot = new Bot(null, user.getId(), title, description, content, now, now);

    botMapper.insert(bot);
    map.put("error_message", "success");

    return map;
}
}

```

先通过前端传过来的 data, 通过 map 的 get 方法获取到 description, title, content 等信息, 然后判断是否合法, 比如: 标题不能为空, 标题长度不能大于 100, 代码不能为空等等, 最后将新的 Bot 添加 数据库中。

```

@RestController
public class AddController {
    @Autowired
    private AddService addService;

    @PostMapping("/user/bot/add/")
    public Map<String, String> add(@RequestParam Map<String, String> data) {
        return addService.add(data);
    }
}

```

前端通过 ajax 调用这个接口传入参数, 后端再返回。

3.8 删除 AI 代码

功能描述: 删除数据库中 bot 的信息, 即删除 AI 代码

实现与描述: 首先需要连接数据库的实体类 bot 如下:

```

@Data
@NoArgsConstructor
@AllArgsConstructor
public class Bot {
    @TableId(type = IdType.AUTO)
    private Integer id;
    private Integer userId;
    private String title;
    private String description;
    private String content;
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai")
    private Date createtime;
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai")
    private Date modifytime;
}

```

解释: Bot 类对应 数据库中的 bot 表 @TableId(type = IdType.AUTO) 表示 id 自增@Data @NoArgsConstructor @AllArgsConstructor 三个注解表示 lombok 这个仓库通过反射机制自动生成 get set toString() 方法 @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "Asia/Shanghai") 表示以什么格式表示日期这个表示先年再月最后天 几点几分, 如 2023-06-15 21:15, timezone 表示时间是以亚洲上海为标准。

写一个 Service

```

public interface RemoveService {
    Map<String, String> remove(Map<String, String> data);
}

```

然后 实现 Impl

```

@Service
public class RemoveServiceImpl implements RemoveService {
    @Autowired
    private BotMapper botMapper;

    @Override
    public Map<String, String> remove(Map<String, String> data) {

```

```

        UsernamePasswordAuthenticationToken authenticationToken =
            (UsernamePasswordAuthenticationToken)
SecurityContextHolder.getContext().getAuthentication();
        UserDetailsImpl loginUser = (UserDetailsImpl)
authenticationToken.getPrincipal();

```

```

        User user = loginUser.getUser();
        int bot_id = Integer.parseInt(data.get("bot_id"));
        Bot bot = botMapper.selectById(bot_id);

        Map<String, String> map = new HashMap<>();
        if (bot == null) {
            map.put("error_message", "Bot 不存在或已被删除");
            return map;
        }
        if (!bot.getUserId().equals(user.getId())) {
            map.put("error_message", "没有权限删除该 Bot");
            return map;
        }
        botMapper.deleteById(bot_id);
        map.put("error_message", "success");
        return map;
    }
}

```

先通过前端传过来的 data, 通过 map 的 get 方法获取到 bot_id, 判断是不是当前用户, 如果 bot 不是空, 并且是当前用户, 通过 botMapper 的 delete 删除数据库中的一个数据

```

@RestController
public class RemoveController {

    @Autowired
    private RemoveService removeService;

    @PostMapping("/user/bot/remove/")
    public Map<String, String> remove(@RequestParam Map<String, String> data) {
        return removeService.remove(data);
    }
}

```

前端通过 ajax 调用这个接口传入参数, 后端再返回。

3.9 其他设计与实现描述

- (1) 地图设计: 为了游戏双方公平, 在设计地图时, 双方是对称的, 并且双方一定有一条路径可以走, 算法用的的 Bfs。算法描述: 从一个点出发判断是不是可以走到另一个点。

- (2) 蛇的设计：为了游戏的刺激性,我们让蛇可以变长,变长算法是：前 10 秒每个一秒长度加一,10 秒后,判断是不是 $\%3 == 0$,如果等于 0,则长度加一,否则长度不变。
- (3) 页面设计：在匹配成功之后,双方如何判断是在左下角还是右上角,这里我们在前端通过 ajax 和 js 拿到双方的用户 id 和双方 a 玩家和 b 玩家进行判断,如果是 a 玩家,则在左下角,否则在右上角。同时在页面上正下方显示出来。这里还需要注意 js 判断类型,需要将双方都用 parseInt 转成 Int,还有判断要用 $===$ 不能用 $==$ 。

4 系统使用说明和测试

4.1 启动流程

SpringBoot 启动流程：首先从 main 找到 run() 方法，在执行 run() 方法之前 new 一个 SpringApplication 对象, 进入 run() 方法，创建应用监听器 SpringApplicationRunListen 开始监听然后加载 SpringBoot 配置环境 (ConfigurableEnvironment), 然后把配置环境 (Environment) 加入监听对象中然后加载应用上下文 (ConfigurableApplicationContext), 当做 run 方法的返回对象最后创建 Spring 容器, refreshContext(context), 实现 starter 自动化配置和 bean 的实例化等工作。

Vue 启动流程：如果一开始没用过 Vue，需要先下载 node 和 npm, 下载完成后，打开 powershell 或 gitbash，输入 vue ui，进入自己的目录导入，启动。如果从 github 上克隆的还需要 npm install，下载完成后同上。

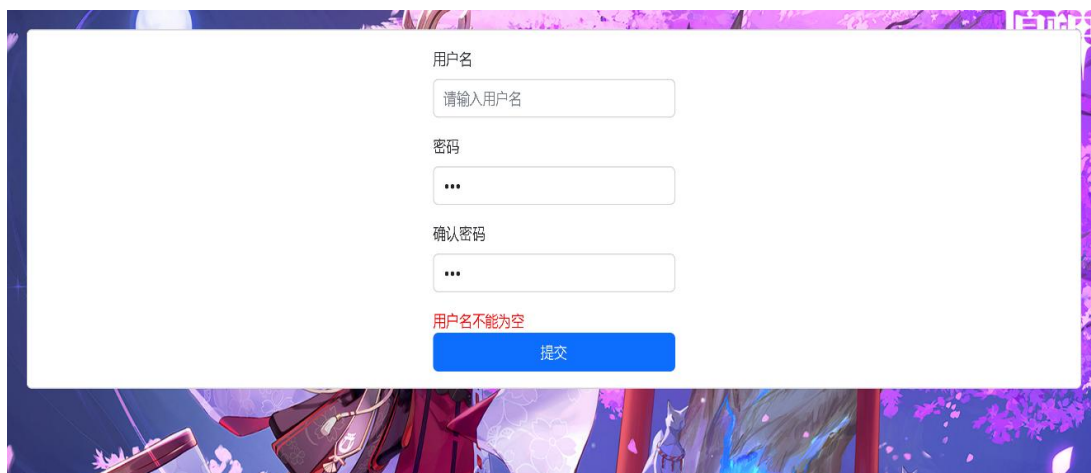
4.2 登录注册功能测试

- (1) 在注册时, 两次密码不一致, 预计结果提示两次密码不一样。测试用例：用户名 zihuia 密码 123 再次输入密码 234。



图 4.1 注册测试图

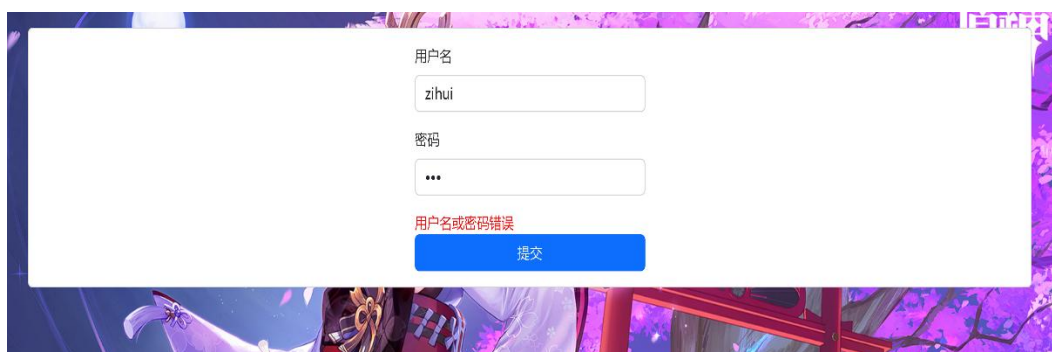
- (2) 在注册时, 不输入用户名, 预计结果提示用户名不能为空。测试用例: 用户名 null 密码 123 再次输入密码 123。



The image shows a registration form with three input fields: '用户名' (Username), '密码' (Password), and '确认密码' (Confirm Password). The '用户名' field contains the placeholder text '请输入用户名'. Below the fields, a red error message '用户名不能为空' is displayed. A blue '提交' (Submit) button is at the bottom.

图 4.2 注册测试图

- (3) 在登录时, 输入密码错误, 预计结果提示用户名或密码错误。测试用例: 用户名 zihui 密码 123。



The image shows a login form with two input fields: '用户名' (Username) and '密码' (Password). The '用户名' field contains the text 'zihui'. The '密码' field contains three dots. Below the fields, a red error message '用户名或密码错误' is displayed. A blue '提交' (Submit) button is at the bottom.

图 4.3 登录测试图

- (4) 在登录时, 输入正确的用户名和密码, 预计结果跳转匹配页面。测试用例: 用户名 zihui 密码

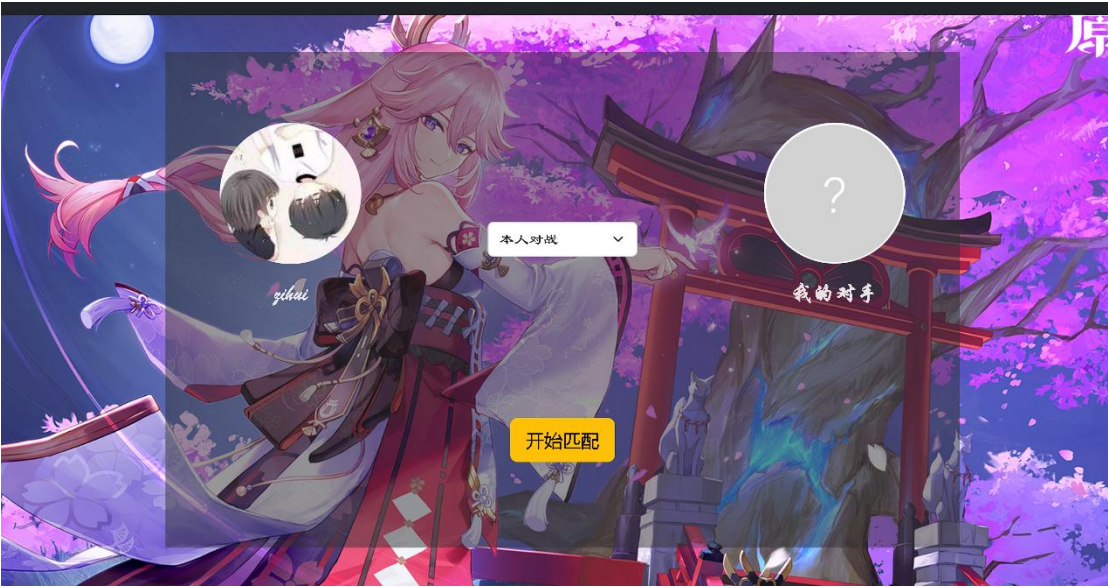


图 4.4 登录测试图

4.3 录像功能测试

在登录成功后, 点击对局列表, 预计结果跳转对局列表页面。

A	B	对战结果	对战时间	操作
zihui	pydmy7	平局	2023-06-15 11:24:16	查看录像
zihui	itoney	A胜	2023-06-01 16:29:20	查看录像
zihui	itoney	B胜	2023-06-01 16:29:01	查看录像
zihui	itoney	A胜	2023-06-01 16:28:25	查看录像
zihui	itoney	A胜	2023-06-01 16:27:31	查看录像
itoney	zihui	B胜	2023-06-01 16:27:05	查看录像
itoney	zihui	B胜	2023-06-01 16:26:34	查看录像
zihui	itoney	A胜	2023-06-01 16:26:07	查看录像
itoney	zihui	B胜	2023-06-01 16:25:42	查看录像
zihui	456	B胜	2023-05-30 17:00:08	查看录像

« 1 2 »

图 4.5 录像功能测试图

然后点击查看录像，预计结果出现录像页面。

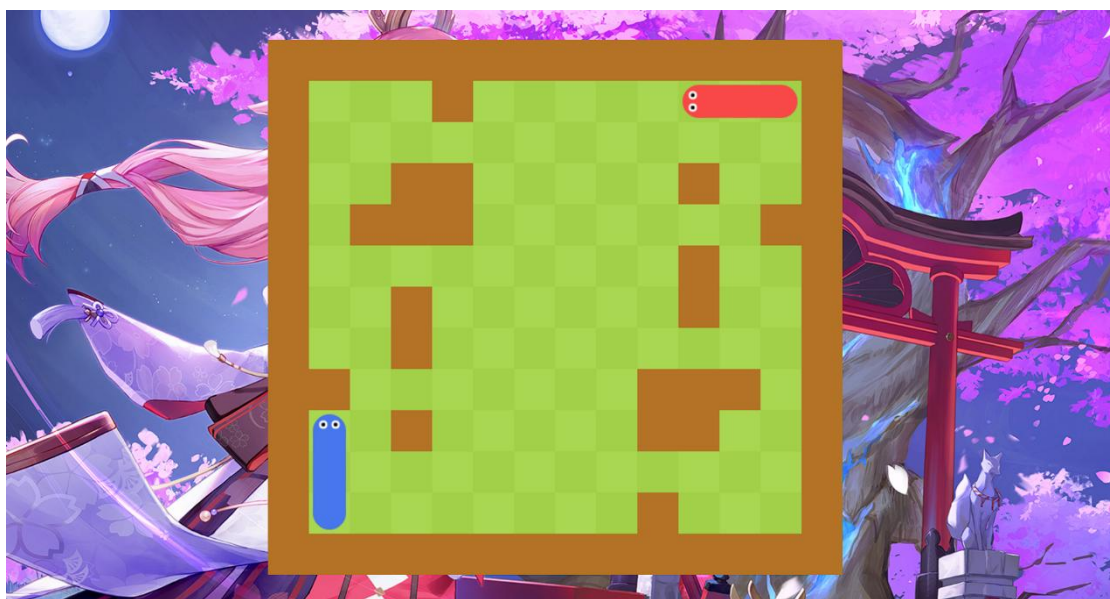


图 4.6 录像功能测试图

4.4 匹配功能测试

(1) 在匹配页面，点击匹配，如果没有其他人匹配, 预计结果一直等待匹配。

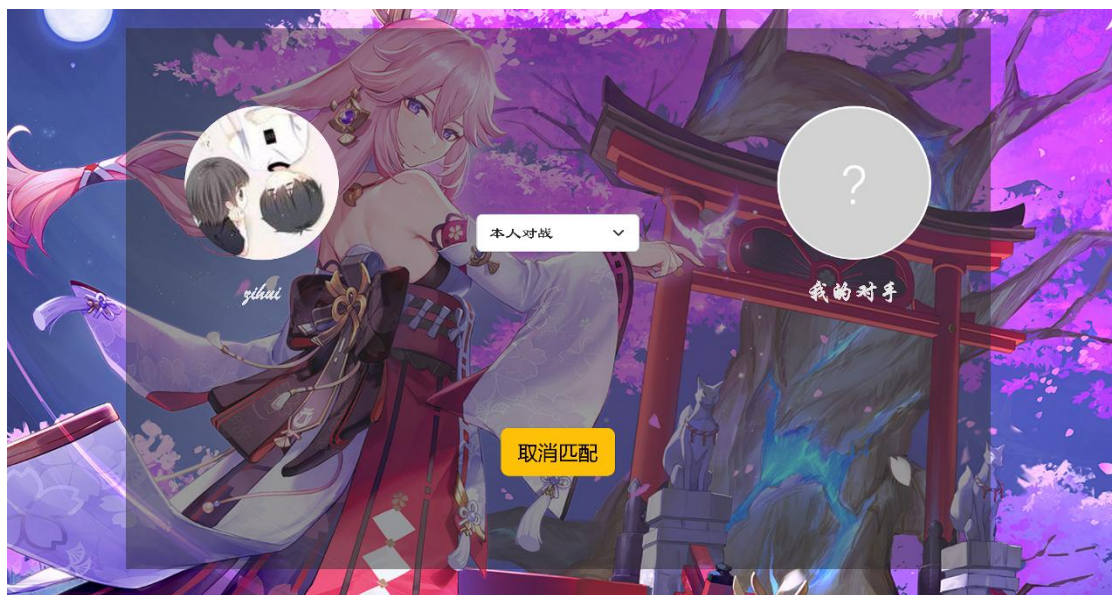


图 4.7 匹配功能测试图

(2) 在匹配页面，点击匹配，如果没有其他人匹配, 预计结果进入对战页面。

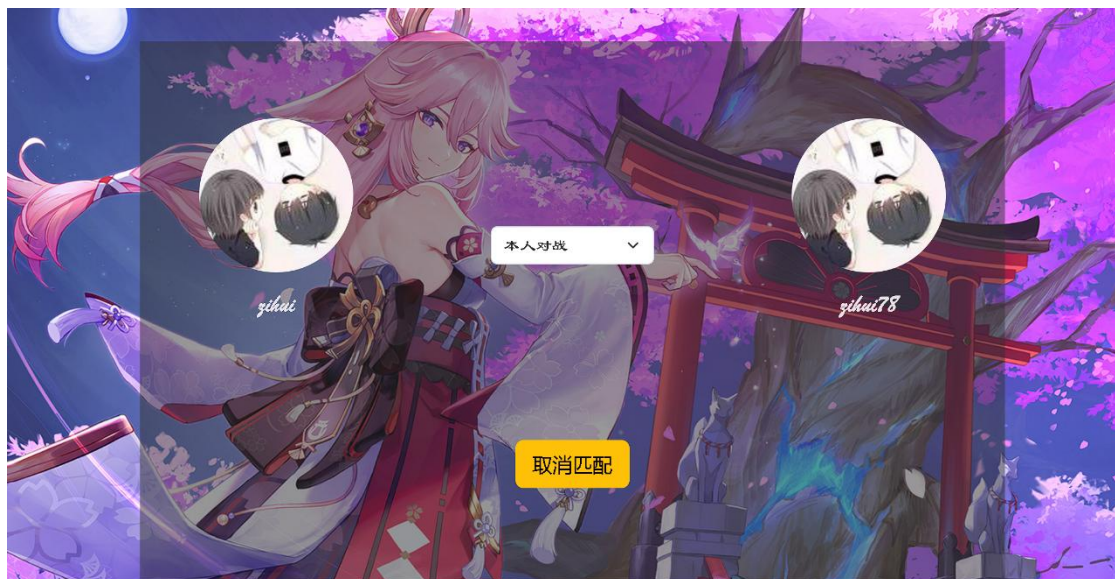


图 4.8 匹配功能测试图



图 4.9 匹配功能测试图

4.5 AI 代码执行功能测试

进入匹配页面后, 在本人对战框框可以选择执行那个 AI 代码。

(1) 选择一个错误的 AI 代码, 预计蛇不会动, 游戏惜败。

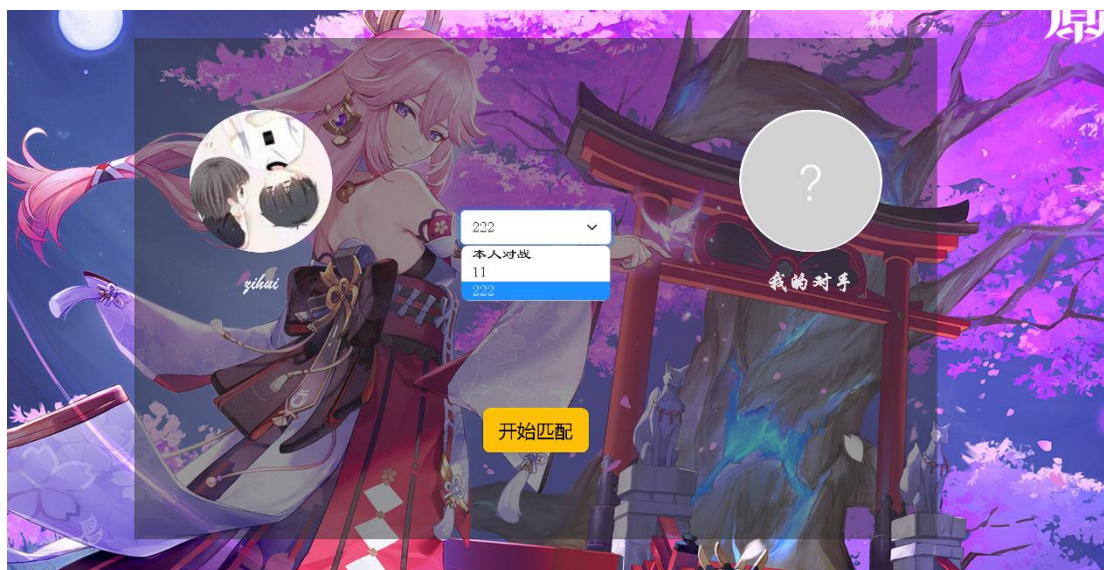


图 4.10 AI 代码执行功能测试图



图 4.11 AI 代码执行功能测试图

(2) 选择一个正确的 AI 代码, 预计蛇动, 到最后双方有一个游戏惜败。



图 4.12 AI 代码执行功能测试图

4.6 删除 AI 功能测试

(1) 在个人页面, 删除 AI 代码, 预计结果删除成功

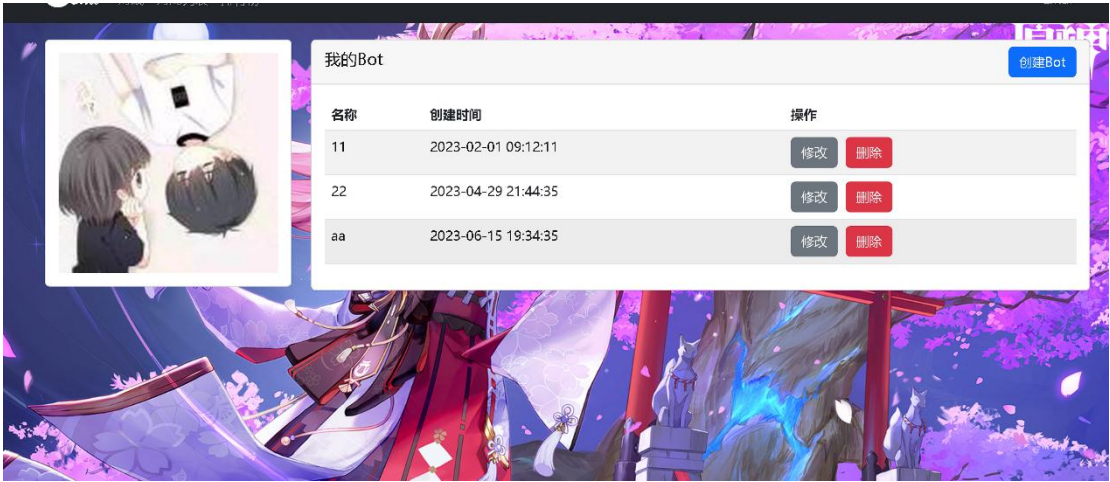


图 4.13 删除 AI 功能测试图

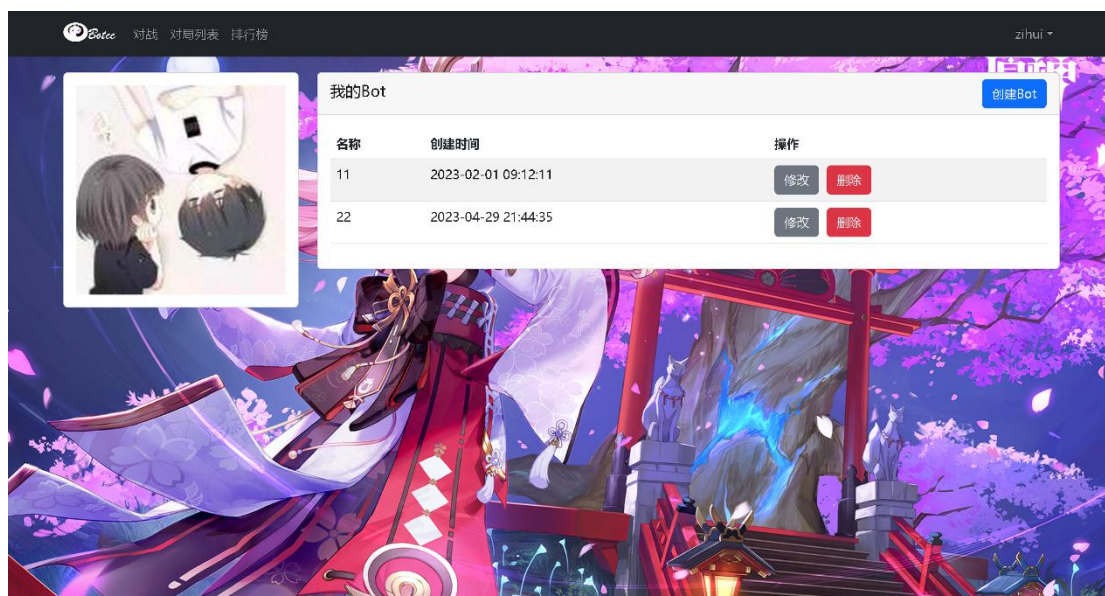


图 4.14 删除 AI 功能测试图

4.7 添加 AI 功能测试

- (1) 在个人页面，添加 AI 代码，如果标题为空,预计结果提示标题不能为空。测试用例：名称 null 描述 aa 代码 retutn 0。

创建Bot

名称
请输入Bot名称

描述
aa

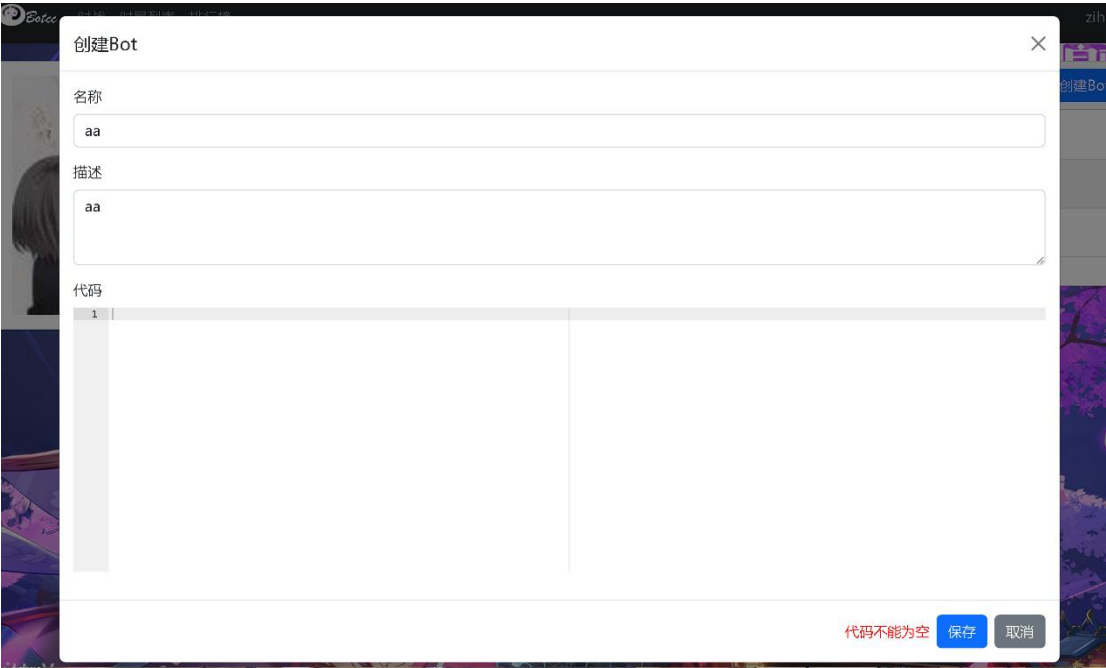
代码
1 return 0

标题不能为空 [保存](#) [取消](#)

图 4.15 添加 AI 功能测试图

- (2) 在个人页面，添加 AI 代码，如果代码为空,预计结果提示代码不能为空。测试用例：

名称 aa 描述 aa 代码 null。



创建Bot

名称
aa

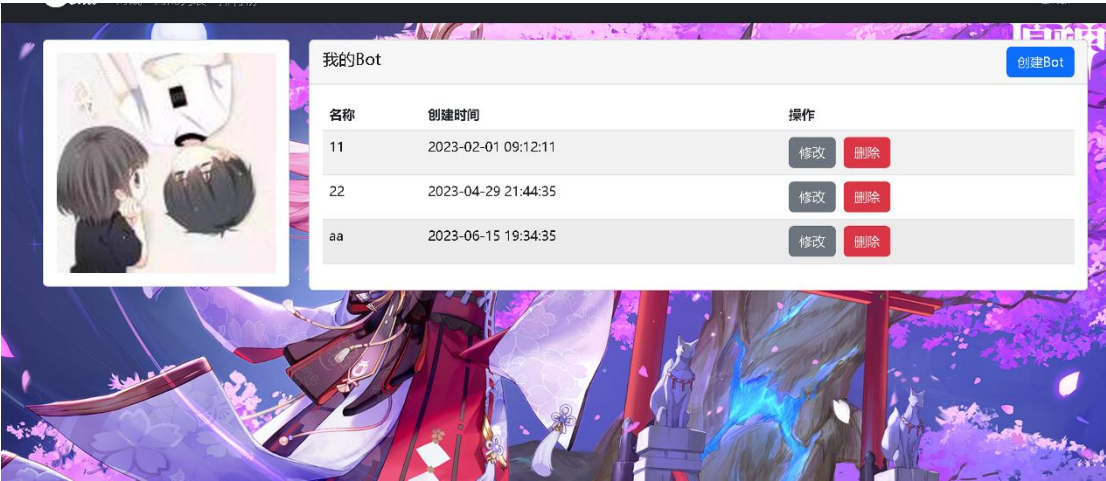
描述
aa

代码
1

代码不能为空 保存 取消

图 4.16 添加 AI 功能测试图

(3) 在个人页面，添加 AI 代码，如果标题不为空,代码不为空,预计结果添加成功。



名称	创建时间	操作
11	2023-02-01 09:12:11	修改 删除
22	2023-04-29 21:44:35	修改 删除
aa	2023-06-15 19:34:35	修改 删除

图 4.17 添加 AI 功能测试图

4.8 更新 AI 功能测试

(1) 在个人页面，更新 AI 代码，如果删除标题,预计结果提示标题不能为空。测试用例：名称 null 描述 aa 代码 retutn 0。

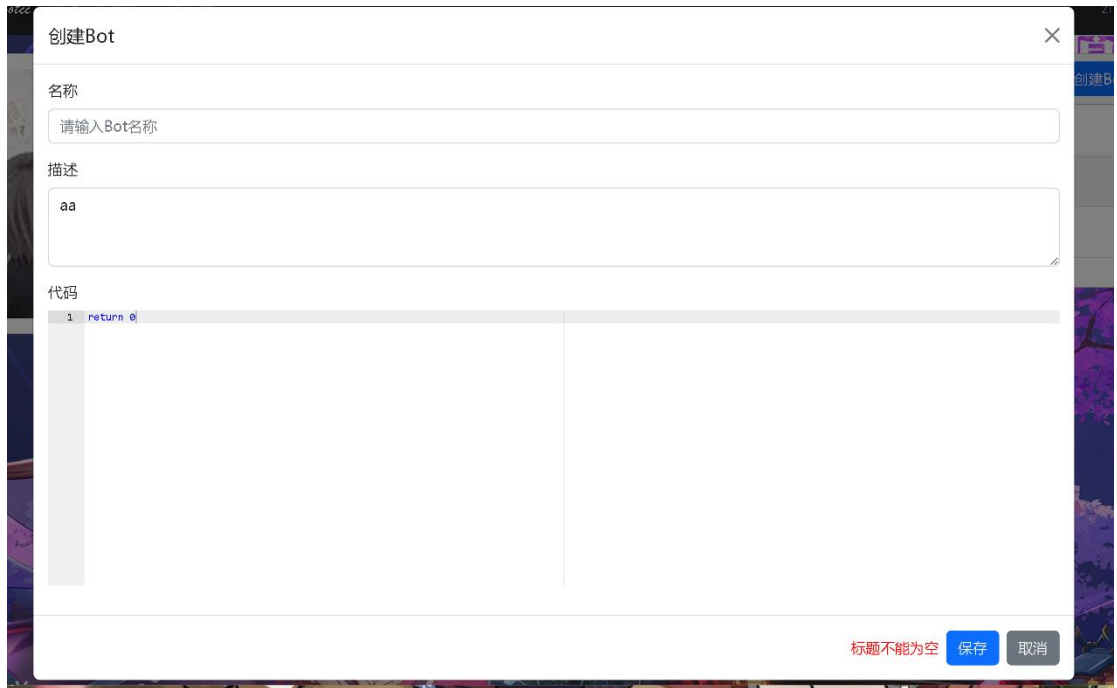


图 4.18 更新 AI 功能测试图

- (2) 在个人页面，更新 AI 代码，如果删除代码,预计结果提示代码不能为空。测试用例：
名称 aa 描述 aa 代码 null。

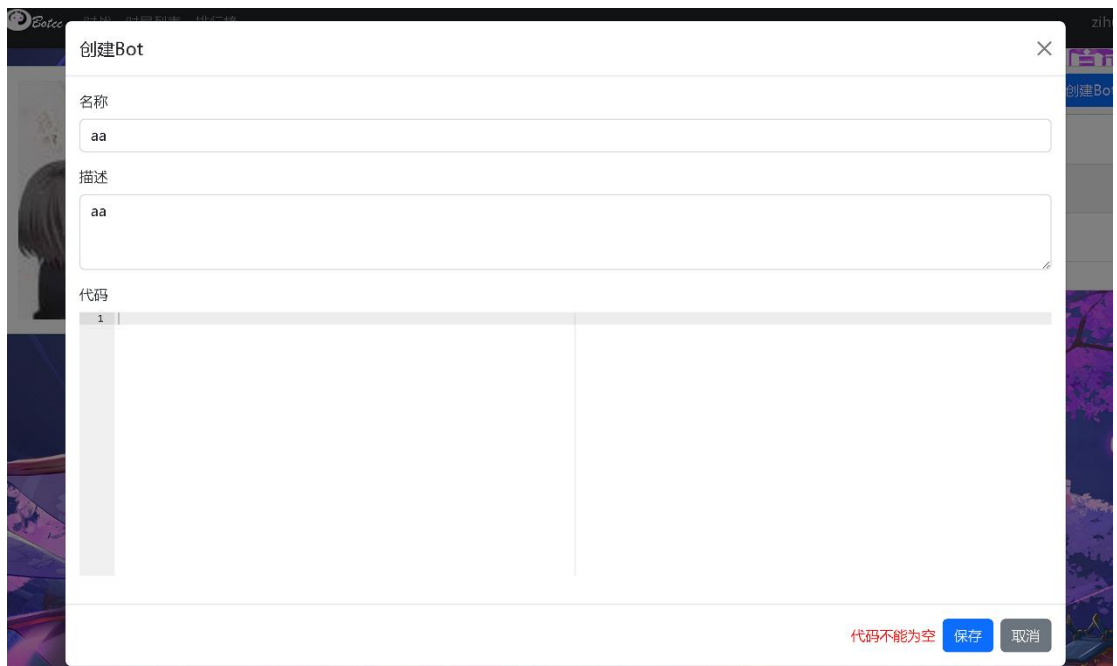


图 4.19 更新 AI 功能测试图

- (3) 在个人页面，更新 AI 代码，如果修改标题,修改代码,预计结果更新成功。

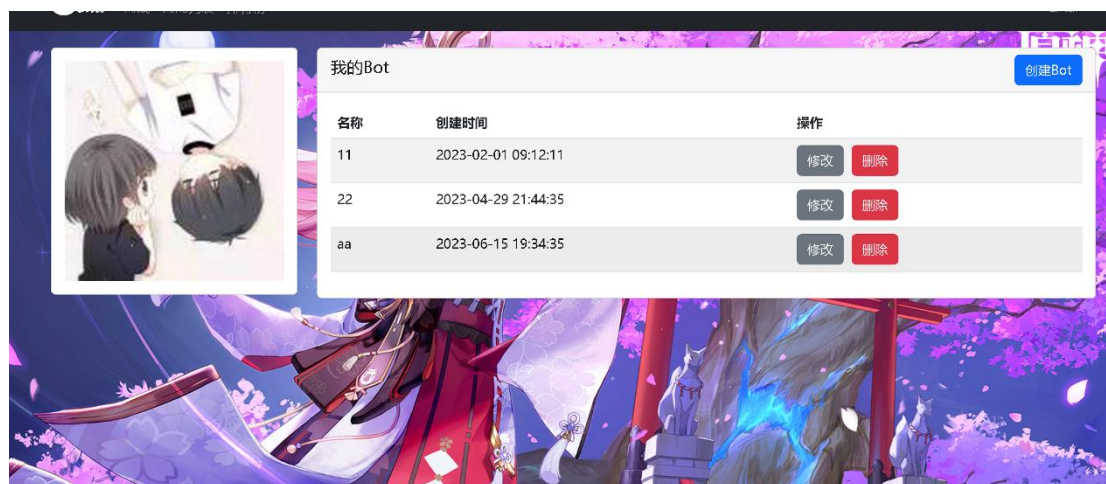


图 4.20 更新 A1 功能测试图

5 问题及解决办法

5.1 JWT - token 如何解决跨域问题：

JSON Web Token（缩写 JWT），是目前最流行的一个跨域认证解决方案：客户端发起用户登录请求，服务器端接收并认证成功后，生成一个 JSON 对象（如下所示），然后将其返回给客户端。客户端再次与服务器端通信的时候，把这个 JSON 对象捎带上，作为前后端互相信任的一个凭证。服务器端接收到请求后，通过 JSON 对象对用户身份进行鉴定，这样就不再需要保存任何 session 数据了。

一般流程是下面这样：

- 1、用户向服务器发送用户名和密码。
- 2、服务器验证通过后，在当前对话（session）里面保存相关数据，比如用户角色、登录时间等等。
- 3、服务器向用户返回一个 session_id，写入用户的 Cookie。
- 4、用户随后的每一次请求，都会通过 Cookie，将 session_id 传回服务器。
- 5、服务器收到 session_id，找到前期保存的数据，由此得知用户的身份。

这种模式的问题在于，扩展性（scaling）不好。单机当然没有问题，如果是服务器集群，或者是跨域的服务导向架构，就要求 session 数据共享，每台服务器都能够读取 session。举例来说，A 网站和 B 网站是同一家公司的关联服务。现在要求，用户只要在其中一个网站登录，再访问另一个网站就会自动登录，请问怎么实现。一种解决方案是 session 数据持久化，写入数据库或别的持久层。各种服务收到请求后，都向持久层请求数据。这种方案的优点是架构清晰，缺点是工程量比较大。另外，持久层万一挂了，就会单点失败。另一种方案是服务器索性不保存 session 数据了，所有数据都保存在客户端，每次请求都发回服务器。JWT 就是这种方案的一个代表。

5.2 Java 是如何解析的 JSON 的：

Java 中并没有内置 JSON 的解析，因此使用 JSON 需要借助第三方类库。经过总结，一共 4 种方式，下面围绕每种方式生成和解析进行阐述：

1. JSON 官方
2. GSON：谷歌开发的 JSON 库，功能十分全面。
3. FastJSON：阿里巴巴开发的 JSON 库，性能十分优秀。
4. Jaskon：社区十分活跃且更新速度很快。

5.3 Redis 是如何解决缓存：

为了提高系统的承压能力和响应速度，可以使用 redis 缓存数据和页面静态化为什么要使用缓存？

缓存数据存储于代码中,而代码运行在内存中,内存的读写性能远高于磁盘,缓存可以大大降低用户访问并发量带来的服务器读写压力;实际开发过程中,企业的数据量,少则几十万,多则几千万,这么大数据量,如果没有缓存来作为“避震器”,系统是几乎撑不住的,所以企业会大量运用到缓存技术;但是缓存也会增加代码复杂度和运营的成本,查询、添加缓存缓存模型和思路,标准的操作方式就是查询数据库之前先查询缓存,如果缓存数据存在,则直接从缓存中返回,如果缓存数据不存在,再查询数据库,然后将数据存入 redis。

更新缓存更新策略:缓存更新是 redis 为了节约内存而设计出来的一个东西,主要是因为内存数据宝贵,当我们向 redis 插入太多数据,此时就可能会导致缓存中的数据过多,所以 redis 会对部分数据进行更新,或者把他叫为淘汰更合适。

内存淘汰:redis 自动进行,当 redis 内存达到咱们设定的 max-memory 的时候,会自动触发淘汰机制,淘汰掉一些不重要的数据(可以自己设置策略方式)超时剔除:当我们给 redis 设置了过期时间 ttl 之后,redis 会将超时的数据进行删除,方便咱们继续使用缓存

主动更新:我们可以手动调用方法把缓存删掉,通常用于解决缓存和数据库不一致问题。

6 项目总结

本文所设计开发的匹配游戏，实现了匹配游戏的一些基本和必需的功能，如游戏的匹配功能，游戏对局录像的功能，。但游戏功能还存在着很多不足,比如对局游戏的难度功能,是不是可以选择不同的难度,录像功能是不是可以搜索自己的对局记录。但经过运行测试,该游戏运行流畅、稳定。可以满足大部分游戏玩家的需求。同时我们通过这个的课程设计,我们对 SpringBoot 和 SpringCloud 的理解进一步加深，进一步理解 前后端 是如何通信的，通过用 MybatisPlus 的使用,对数据库的应用有了进一步的提高。

参考文献

- [1] 尤雨溪 《vue 开发》
- [2] spring 《SpringBoot》 <https://spring.io/>
- [3] MySQL 《MyBatisPlus》 <https://baomidou.com/>
- [4] Java 《Java》 <https://docs.oracle.com/javase/8/docs/api/>